

Open ACE

自托管 AI Coding Agent 工作台与治理控制面

把团队已在用的多个 AI 编码工具接进来，把密钥、成本和风险统一管起来。

Apache 2.0 开源

v1.2.0

支持私有化部署

当 AI 编码工具进入团队，问题才刚刚开始



密钥失控

API Key 散落在个人电脑和远程机器，泄漏风险高、轮换难、无法追溯



远程难跑

AI CLI 需要跑在内网开发机、测试机、GPU 机器，反复传 SSH 凭据很笨重



工具分散

Claude Code、Qwen Code、Codex、OpenClaw 各自为战，无统一入口与历史



成本盲区

团队花了多少 Token、多少钱、谁在用、是否超支——基本是黑盒



审计缺位

合规、审计、配额、敏感内容检测缺失，上不了生产环境



价值难证

AI 投入产出比说不清，效率提升无法量化，决策无依据

一个平台，双重价值

Open ACE = AI 编码工作台 + AI 治理控制面

开发者在浏览器里统一使用 Claude Code、Qwen Code、Codex、OpenClaw 等 AI 编码工具，并把它们跑在团队自己的远程机器上；

管理员则集中管理 API Key、权限、成本、配额、审计和合规——一切自托管、数据不出企业网络。



统一入口

多个 AI CLI 一个浏览器工作台，统一会话、历史与治理



远程执行

Remote Agent 让 AI CLI 直接在目标机器运行，免反复传凭据



密钥安全

API Key 加密留服务端，只下发短期代理令牌给会话



治理可视

Token、成本、配额、审计、合规、ROI 一屏掌握

两种模式，服务两类用户

Work 模式 — 让 AI 成为超级助手

面向每一位开发者，提供流畅的 AI 交互体验

- 多 AI 工具集成：Claude Code / Qwen Code / Codex / OpenClaw / ZCode 统一入口
- 远程工作区：浏览器里选远程机器，启动 AI 编码会话、终端、目录浏览
- 智能会话管理：历史记录、会话恢复、上下文记忆、跨工具同步
- 提示词库：团队共享优质提示词，最佳实践一键复用
- 快速检索：跨会话搜索历史对话，知识沉淀不丢失

Manage 模式 — 让 AI 治理有据可依

面向管理者，提供全方位的 AI 使用洞察与管控

- 用量可视化：Token 消耗趋势、成本分析、使用热力图一目了然
- API Key 治理：加密存储密钥，通过代理令牌调用，避免密钥下发
- 智能告警：配额预警、异常检测、超支提醒，风险早知道
- 合规审计：敏感内容检测、对话追溯、合规报告与 CSV 下载
- 多租户 + ROI：部门隔离、权限控制、投入产出比量化

八大核心能力，覆盖 AI 编码全生命周期



Agent 控制面

统一管理工具、用户、机器、密钥、配额、成本和审计，而非替代每个 AI CLI



多 CLI 适配器

原生适配 Claude Code、Qwen Code、Codex、OpenClaw、ZCode，含会话恢复与权限模式



Remote Agent

AI CLI 直接在 Linux/macOS/Windows 远程机器运行，支持浏览器终端与 VSCode 代理



API Key 代理

真实密钥加密存服务端，远程 Agent 只拿短期代理令牌，统一配额与用量统计



AI 自主开发

端到端工作流：规划→开发→PR 审查→报告，支持重试 /Fork/ 里程碑，可跑在 gVisor/Kata 沙箱



Model Gateway

兼容 LiteLLM 模型网关，厂商中立，可对接集中式模型网关并保留配额与归因



云原生部署

Kubernetes Kustomize 一键部署，3 副本 + HPA + Prometheus 指标，兼容 OpenShift



企业集成

SSO / 飞书 / 钉钉、多租户权限模型、Kubernetes、反向代理

Remote Agent + API Key 代理：安全与效率兼得

浏览器 (React SPA)

Work 模式 (全员) · Manage 模式 (管理员)



Flask API 服务 (自托管)

23 Blueprints · 密钥加密存储 · 配额 / 审计 / 合规 · API Key 代理签发短期令牌



Remote Agent (远程机器守护进程)

HTTP 轮询 · CLI 子进程 · WebSocket PTY 终端 · 会话同步 (Claude/Qwen/Codex/OpenClaw)

为什么这样设计？

- 真实 API Key 永不出服务端
- 远程机器只持有短期代理令牌
- 令牌可配额、可回收、可追溯
- 支持内网 / 私有网络机器
- 终端断线可恢复屏幕
- 厂商中立，可接 LiteLLM

✦ 适合需要私有化部署、内网远程机器、统一密钥代理、团队配额和可追溯审计的研发与 IT 团队

Kubernetes / OpenShift 云原生集成

平台部署：K8s 高可用

- Kustomize 一键部署 (kubectl apply -k k8s/)
- 3 副本 + HPA 自动扩缩容 (3→10 Pod)
- PostgreSQL / Redis StatefulSet 持久化
- /livez /readyz 探针 + Prometheus /metrics
- PDB + Pod 反亲和 + 优雅滚动更新
- NetworkPolicy 最小权限 + 非 root 运行



AI 自主开发：沙箱隔离

- 三后端按租户 / 项目选择：POSIX / 远程机 / 沙箱
- gVisor / Kata 双档安全容器运行时
- 非 root + 只读根文件系统 + seccomp 强化
- CPU / 内存 / PIDs / 存储限额 + TTL
- 默认拒绝出站 + FQDN 白名单
- Fail-closed：隔离不达标即拒绝并审计



多用户工作区：资源隔离

- 每用户 WebUI 独立沙箱 Pod，Token 身份
- 无宿主 OS 账号 / sudo 链 / 宿主凭据
- 租户级 OS 隔离：跨租户访问阻断、可回收
- UID 跨容器重建固定，配额 60 秒巡检执行
- 会话历史快照随 Pod 回收与恢复

🖥️ OpenShift / OKD 兼容：标准 Kustomize 清单，安全上下文符合 restricted SCC 约束，SSRF 防护覆盖 OpenShift 内部 DNS

对开发者、管理者、企业三个层面创造价值



对开发者

- 一个入口用遍所有 AI 编码工具
- 远程机器随时可用，免配环境
- 会话历史不丢，知识可沉淀
- 专注编码，密钥与环境由平台托管



对管理者

- Token / 成本 / 配额一屏可视
- 告警与异常检测，风险早知道
- 合规审计与报告导出，上得了生产
- ROI 量化，AI 投入产出说得清



对企业

- 数据与密钥不出企业网络
- 私有化部署，满足合规要求
- 多租户隔离，部门独立核算
- 开源可控，无厂商锁定

与市场主流方案的差异化定位

能力维度	Open ACE	通用 LLM 网关 (LiteLLM 等)	AI IDE/ 插件 (Cursor 等)	企业自建脚本
定位	工作台 + 治理控制面	模型路由 / 计费	单人编码体验	临时拼装
自托管 / 私有化	✔️ 默认自托管	✔️	❌ 多为云服务	✔️
多 AI CLI 统一入口	✔️ Claude/Qwen/Codex 等	❌ 仅模型层	⚠️ 各自封闭	❌
Remote Agent 远程执行	✔️ 浏览器内远程机器	❌	❌	⚠️ 需自建
API Key 不下发	✔️ 短期代理令牌	⚠️ 依赖网关	❌ 密钥在本机	❌
成本 / 配额 / 审计 / 合规	✔️ 一体化治理	⚠️ 仅计费	❌	❌
多租户 / SSO / 飞书	✔️	⚠️ 部分	❌	❌
K8s 原生 & AI 沙箱隔离	✔️ HPA+gVisor/Kata 沙箱	⚠️ 可容器化	❌	❌ 需自建
开源协议	Apache 2.0	多协议	闭源	—

为什么是 Open ACE ?



默认自托管，数据边界可控

密钥与会话数据留在企业自己的网络与数据库，满足私有化与合规要求，无云依赖。



控制面而非又一个 CLI

不替代任何 AI 编码工具，而是把团队已在用的工具纳入统一的入口、权限、成本与审计体系。



短期代理令牌架构

真实 LLM API Key 加密留服务端，远程会话仅持有时效令牌，可配额、可回收、可追溯。



Remote Agent 真实远程执行

AI CLI 直接在开发 / 测试 / GPU 机器运行，浏览器内终端 + VSCode 代理，免反复传 SSH。



云原生 + 双层沙箱隔离

K8s 高可用部署；AI 自主开发与多用户工作区运行在 gVisor/Kata 沙箱，故障关闭式安全。



厂商中立 + 多租户

兼容 LiteLLM 模型网关，多 CLI 统一治理，部门隔离与独立核算。

谁最需要 Open ACE ?

研发团队

已在用 Claude Code / Qwen Code / Codex , 需要统一入口、共享提示词与会话沉淀。

AI 平台团队

构建内部 AI 工具平台, 需要密钥代理、远程执行与统一治理控制面。

DevOps / IT 团队

负责自托管基础设施、审计与访问控制, 需要私有化部署与多租户。

安全合规团队

不能用公共 Demo、不能让密钥散落, 需要审计追踪与合规报告导出。

GPU / 算力团队

AI CLI 需跑在 GPU 机器或测试环境, 需要浏览器内远程工作区与终端。

多部门企业

需要部门隔离、独立配额核算、SSO 与飞书 / 钉钉集成。

5 分钟启动，渐进式落地

一键部署 (推荐)

```
git clone https://github.com/open-ace/open-ace.git
cd open-ace
docker compose up -d --build

# 访问 http://localhost:19888
```

其他部署方式

- 源码安装 (本地开发)
- Kubernetes Kustomize 部署 (3 副本 + HPA + Prometheus)
- Linux systemd / macOS launchd 远程 Agent
- 支持 SQLite (单机) 与 PostgreSQL (生产)

渐进式落地路线

- Step 1** 本地 Docker 试用，评估 Work / Manage 两模式
- Step 2** 接入团队 API Key，开启 API Key 代理
- Step 3** 部署 Remote Agent 到开发 / 测试 / GPU 机器
- Step 4** 上 K8s 高可用部署，开启 AI 沙箱隔离
- Step 5** 配置配额、审计、SSO，产出合规与 ROI 报告

把 AI Coding Agent 接进来， 把密钥、成本和风险管起来。

Open ACE — self-hosted workspace and control plane for AI coding agents.



官网

www.open-ace.com



代码仓库

github.com/open-ace/open-ace



文档

open-ace.github.io/open-ace-docs



反馈

[GitHub Discussions](https://github.com/open-ace/open-ace/discussions)